

REFERENCE

GATITAS TAPABAÑOS NACATLÁN



Última actualización: 2026-08-23

Índice

- 1. C++ 2
 - 1.1. Configuración 2
 - 1.1.1. Compilación 2
 - 1.1.2. Plantilla 2
 - 1.2. STL (Standard Template Library) 2
 - 1.2.1. Vector 2
 - 1.2.2. Multiset 3
 - 1.2.3. Map 3
 - 1.2.4. Priority Queue 3
 - 1.2.5. Deque 3
 - 1.2.6. Bitset 3
 - 1.3. Funciones para strings 3
 - 1.4. Funciones de <algorithm> 4
 - 1.5. Otras funciones útiles 5
 - 1.6. Constantes 5
- 2. Estructuras de Datos 5
 - 2.1. Policy-based data structures 5
 - 2.1.1. Ordered Set 5
 - 2.2. Segment tree 5
 - 2.3. Fenwick tree 6
- 3. Ordenamiento y Búsqueda 6
 - 3.1. Two Pointers 6
 - 3.2. Búsqueda binaria en funciones monótonas 7
- 4. Teoría de gráficas 7
 - 4.1. Graph Traversal 7
 - 4.1.1. Floodfill 7
 - 4.2. Disjoin Set Union 9
 - 4.3. Topological Sort 10
 - 4.4. Detección de ciclos 11
 - 4.5. Lowest Common Ancestor (LCA) 12
- 5. Programación Dinámica 13
- 6. Teoría de números 14
 - 6.1. Cribas 14
 - 6.2. Euclid's Algorithm 15
 - 6.2.1. Extended Euclid's Algorithm: 15
 - 6.3. Euler's Theorem 15
 - 6.4. Solving Equations 16
- 7. Combinatoria 16
 - 7.1. Binomial Coefficients 16
- 8. Strings 16

8.1.	Trie	16
8.2.	String hashing	16
8.3.	KMP — Knuth-Morris-Pratt	17
9.	Manipulación de bits	18
9.1.	Máscara de bits	18
9.1.1.	Manipulación individual de bits	18
9.1.2.	Working with the rightmost 1-bit	18
9.1.3.	Creating useful bit masks	18
9.1.4.	Common bit manipulation tricks	18
9.1.5.	Bit operations as math alternatives	18
9.1.6.	GNU C++ compiler built-in functions	19
9.2.	Representing Sets	19
10.	Matemáticas	19
10.1.	Operaciones básicas con matrices	19
11.	Teoría	20
11.1.	Teoría de números	20
11.1.1.	Algunas funciones relevantes	20
11.1.2.	Aritmética modular	21
11.2.	Combinatoria	21
11.2.1.	Números de Fibonacci	21
11.2.2.	Números de Catalán	21
11.2.3.	Identidades binomiales	22
11.2.4.	Estrellas y barras	22
11.2.5.	Principio de inclusión-exclusión	23
11.2.6.	Derangement (subfactorial)	23
11.3.	Primeros términos de algunas sucesiones básicas	23
11.4.	Geometría	23
11.4.1.	Ternas pitagóricas	23
11.4.2.	Trigonometría	23
11.4.3.	Producto punto	24
11.5.	Sumas y series	24
12.	Otros	25
12.1.	Estructura para fracciones	25

1. C++

1.1. Configuración

- **Imprimir con n decimales:** `cout << fixed << setprecision(n);`

1.1.1. Compilación

```

cpp/compilation.sh ↗
1 co() {
2     g++ -std=c++20 -O2 -Wall -Wextra -Wshadow -Wconversion -o $1 $1.cpp
3 }
4
5 run() {
6     co $1 && ./$1;
7 }

```

1.1.2. Plantilla

```

cpp/template.h ↗
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define endl '\n'
6 #define dbg(x) cout << #x << ": " << x << endl
7 #define sz(x) int((x).size())
8 #define all(v) v.begin(), v.end()
9 #define rall(v) v.rbegin(), v.rend()
10 #define pb push_back
11 #define mp make_pair
12 #define fi first
13 #define se second
14
15 using ll = long long;
16 const ll INF = 1e18;
17 const ll MOD = 1e9 + 7;

```

1.2. STL (Standard Template Library)

1.2.1. Vector

- $v.insert(it, x) - O(n)$:** Inserta x en la posición apuntada por el iterador it
- $v.erase(it) - O(n)$:** Remueve el elemento apuntado por el iterador it
- $v.erase(it1, it2) - O(n)$:** Remueve los valores en el intervalo $[it1, it2)$ de iteradores

1.2.2. Multiset

En un multiset, **erase** remueve *todas* las copias de un valor, y **count** devuelve el número de copias de un valor.

Remover una sola ocurrencia de un valor x: `s.erase(s.find(x))`

Las funciones **count** y **erase** tienen un factor adicional de $O(k)$ donde k es el número de elementos contados/eliminados. En particular, *no* es eficiente contar el número de copias de un valor en un multiset usando la función **count**.

1.2.3. Map

m.erase(key) – $O(\log n)$: Remueve la clave **key** del mapa

1.2.4. Priority Queue

En la cola de prioridad por defecto el mayor elemento es el primero en salir.

pq.push(x) – $O(\log n)$: Agrega **x** a la cola de prioridad

pq.pop() – $O(\log n)$: Remueve el mayor elemento de la cola de prioridad

pq.top() – $O(1)$: Devuelve el mayor elemento de la cola de prioridad

priority_queue<int, vector<int>, greater<int>> **pq**: Cola de prioridad en la que el elemento más pequeño es el primero en salir

Si solo necesitamos encontrar o eliminar eficientemente el elemento mínimo o máximo, es buena idea usar una cola de prioridad en lugar de un set o multiset.

1.2.5. Deque

Es un arreglo dinámico que puede manipular eficientemente ambos extremos de la estructura.

d.push_back(x) – $O(1)$: Agrega **x** al final

d.push_front(x) – $O(1)$: Agrega **x** al inicio

d.pop_back() – $O(1)$: Remueve el elemento del final

d.pop_front() – $O(1)$: Remueve el elemento del inicio

1.2.6. Bitset

Sirve para representar máscaras de bits cuando un entero no es suficiente. Posee operaciones optimizadas por el procesador.

bitset<N> a – $O(1)$: Crea un bitset de tamaño **N** (**N** debe ser constante)

a[i] – $O(1)$: Retorna el valor del bit en la posición **i**

a.count() – $O(N/w)$: Retorna el número de bits con 1

a.size() – $O(1)$: Retorna el tamaño del bitset

a.set(i) – $O(1)$: Pone el bit en la posición **i** a 1

a.set() – $O(N/w)$: Pone todos los bits a 1

a.reset(i) – $O(1)$: Pone el bit en la posición **i** a 0

a.reset() – $O(N/w)$: Pone todos los bits a 0

a.flip(i) – $O(1)$: Invierte el bit en la posición **i**

a.flip() – $O(N/w)$: Invierte todos los bits

a.to_ulong() – $O(N/w)$: Retorna el valor del bitset como un entero sin signo (si el tamaño del bitset es mayor al número de bits de un entero, solo se consideran los bits menos significativos)

Imprimir un entero como binario: `cout << bitset<8>(mask) << "\n";`

El bitset también soporta operaciones como: **&**, **|**, **^**, **~**, **<<**, **>>**, entre otras

Obs: *w* es el tamaño de palabra del procesador, en general 32 o 64 bits.

1.3. Funciones para strings

s.substr(i, len) – $O(n)$: Retorna el substring que inicia en **i** y tiene longitud **len**. Si **len** se omite va hasta el final.

s.find(str, pos), **s.rfind(str, pos)** – $O(nm)$: Busca la primera (**find**) o última (**rfind**) ocurrencia de **str** en **s** a partir de **pos**. Retorna **string::npos** si no existe.

```
s = "abcabcabc";  
s.find("bc"); // 1  
s.rfind("bc"); // 7
```

s.replace(pos, len, str) – $O(n)$: Reemplaza **len** caracteres desde **pos** con la cadena **str**.

```
s = "hola mundo";  
s.replace(5, 5, "C++"); // "hola C++"  
s.replace(0, 4, "adios"); // "adios C++"
```

s.erase(pos, len), **s.erase(iterator)** – $O(n)$: Elimina **len** caracteres desde **pos**, o el carácter apuntado por un **iterator**. Muy útil para remover caracteres/subcadenas no deseados durante el procesamiento.

```
s = "hola mundo";  
s.erase(4, 6); // "hola"
```

s.insert(pos, str), **s.insert(pos, n, ch)** – $O(n)$: Inserta **str** en la posición **pos**, desplazando el resto. También puede insertar **n** copias de un carácter **ch**.

```
s = "holamundo";  
s.insert(4, " "); // "hola mundo"
```

```
t = "abc";  
t.insert(1, 3, '*'); // "a***bc"
```

s.find_first_of(chars, pos), **s.find_last_of(chars, pos)** – $O(nm)$: Encuentra la posición del primer o último carácter de **s** que pertenezca al conjunto **chars**. Ideal para parsing de múltiples delimitadores en una sola llamada.

```
s = "hola,mundo;cpp";  
s.find_first_of(";," ); // 4 (';')  
s.find_last_of(";," ); // 9 (';')
```

s.compare(str), s.compare(pos, len, str) – $O(n)$: Compara lexicográficamente.

Retorna 0 si son iguales, < 0 si $s < \text{str}$ y > 0 si $s > \text{str}$. Permite comparar subcadenas sin extraerlas, evitando allocaciones innecesarias.

```
"abc".compare("abc") == 0; // igual
"abc".compare("abd") < 0;  // menor
```

```
s = "hola mundo";
s.compare(5, 5, "mundo") == 0; // true
```

count(all(s), ch) – $O(n)$: Cuenta las ocurrencias de un carácter **ch** en el **string**.

```
s = "banana";
count(all(s), 'a'); // 3
count(all(s), 'n'); // 2
count(all(s), 'b'); // 1
```

isalpha(c), isdigit(c), isalnum(c), isspace(c) – $O(1)$: Verifican si un carácter **c** es letra, dígito, alfanumérico o espacio respectivamente.

s.erase(unique(all(s)), s.end()) – $O(n)$: Elimina caracteres duplicados consecutivos. Si se aplica sobre un **string** ordenado, elimina todos los duplicados.

```
s = "aabbccdde";
s.erase(unique(all(s)), s.end());
// "abcde"
```

```
// Todos los duplicados (con sort previo)
t = "bcaabbcc";
sort(all(t)); // "aabbcc"
t.erase(unique(all(t)), t.end()); // "abc"
```

1.4. Funciones de <algorithm>

fill(all(v), x) – $O(n)$: Rellena **v** con **x**

binary_search(begin, end, val) – $O(\log n)$: Retorna **true** si **val** esta en el rango ordenado.

```
if (binary_search(all(v), 5))
    cout << "Existe";
```

next_permutation(begin, end) – $O(n)$: Genera la siguiente permutación lexicográfica. Caso especial: retorna **false** y deja el rango en la primera permutación (la ordenada) si ya era la última.

```
vector<int> v = {3,2,1};
bool hay = next_permutation(all(v));
// hay == false, v = {1,2,3}
```

max_element(begin, end) – $O(n)$: Retorna iterador al maximo. Para varios maximos, retorna el primero.

```
auto it = max_element(all(v));
if (it != v.end())
    cout << *it;
```

min_element(begin, end) – $O(n)$: Retorna iterador al minimo. Para varios mínimos, retorna el primero.

```
int min_val = *min_element(all(v));
// asumiendo no vacio
```

minmax_element(begin, end) – $O(n)$: Retorna **pair<minIt, maxIt>**. Caso especial: con elementos iguales, **min** y **max** pueden apuntar al mismo elemento. Rango vacio -> ambos **end()**.

```
auto [mn, mx] = minmax_element(all(v));
```

count(begin, end, val) – $O(n)$: Cuenta ocurrencias de **val**.

```
int cnt = count(all(v), 2);
// puede ser 0
```

count_if(begin, end, pred) – $O(n)$: Cuenta elementos que cumplen **pred**.

```
int pares = count_if(all(v), [](int x) {
    return x % 2 == 0;
});
```

find(begin, end, val) – $O(n)$: Retorna un iterador para la primera ocurrencia del elemento **x** en **v**, o **v.end()** si no existe

```
auto it = find(all(v), 3);
if (it != v.end())
    cout << "Encontrado en indice " << it - v.begin();
```

rotate(begin, new_begin, end) – $O(n)$: Rota de forma que **new_begin** pasa a ser el primer elemento. Caso especial: Si **new_begin == begin** o **new_begin == end**, no hay rotación efectiva.

unique(begin, end) – $O(n)$: “Compacta” elementos duplicados consecutivos moviendo las primeras ocurrencias al frente. Caso especial: No cambia el tamaño del contenedor. Devuelve iterador al nuevo final lógico; se debe usar **erase** para eliminar los sobrantes.

```
vector<int> v = {1,1,2,2,2,3};
v.erase(unique(all(v)), v.end());
// {1,2,3}
```

merge(b1, e1, b2, e2, out) – $O(n + m)$: Fusiona dos rangos ordenados en uno solo. Caso especial: Si los rangos de entrada no están ordenados, el comportamiento es indefinido.

```
vector<int> a = {1,3,5}, b = {2,4,6};
vector<int> c(6);
merge(all(a), all(b), c.begin());
// {1,2,3,4,5,6}
```

`partition(begin, end, pred) –  $O(n)$ :` Reordena: primero los que cumplen `pred`, luego el resto. Caso especial: No garantiza el orden relativo dentro de cada grupo. Usar `stable_partition` si se necesita estabilidad.

```
auto it = partition(all(v), [](int x) {
    return x % 2 == 0;
});
// pares al frente, impares al final
```

1.5. Otras funciones útiles

`round(x) –  $O(1)$ :` Retorna el entero más próximo a `x`

1.6. Constantes

Constante	Nombre	Valor aproximado
π	M_PI	3.141592...
$\pi/2$	M_PI_2	1.570796...
$\pi/4$	M_PI_4	0.785398...
$1/\pi$	M_1_PI	0.318309...
$2/\pi$	M_2_PI	0.636619...
$2/\sqrt{\pi}$	M_2_SQRTPI	1.128379...
$\sqrt{2}$	M_SQRT2	1.414213...
$1/\sqrt{2}$	M_SQRT1_2	0.707106...
e	M_E	2.718281...
$\log_2 e$	M_LOG2E	1.442695...
$\log_{10} e$	M_LOG10E	0.434294...
$\ln 2$	M_LN2	0.693147...
$\ln 10$	M_LN10	2.302585...

2. Estructuras de Datos

2.1. Policy-based data structures

The `g++` compiler also supports some data structures that are not part of the C++ standard library. Such structures are called policy-based data structures. To use these structures, the following lines must be added to the code:

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
```

2.1.1. Ordered Set

We can define a data structure `indexed_set` that is like set but can be indexed like an array. The definition for `int` values is as follows:

```
using ordered_set = tree<
    int,
    null_type,
    less<int>,
    rb_tree_tag,
    tree_order_statistics_node_update
>;
```

This data structure supports all the operations as a set in C++ in addition to the following:

- `find_by_order(k)`: returns an iterator to the k -th smallest element (0-based indexing)
- `order_of_key(x)`: returns the number of elements in the set that are strictly less than x

2.2. Segment tree

- TIME
- $O(n)$ for building
 - $O(\log n)$ for queries and updates

estructuras/segment-tree.cpp

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const int MAXN = 2e5;
7  ll a[MAXN];
8  struct segtree {
9      int n;
10     vector<ll> tree;
11
12     segtree(int n) : n(n) {
13         tree.resize(4 * n);
14         build(1, 0, n - 1);
15     }
16
17     void build(int v, int lx, int rx) {
18         if (lx == rx) {
```

```

19         tree[v] = a[lx];
20         return;
21     }
22
23     int m = (lx + rx) / 2;
24     build(2 * v, lx, m);
25     build(2 * v + 1, m + 1, rx);
26     tree[v] = tree[2 * v] + tree[2 * v + 1];
27 }
28
29 void update(int idx, int new_val, int v, int lx, int rx) {
30     if (lx == rx) {
31         tree[v] = a[idx] = new_val;
32         return;
33     }
34
35     int m = (lx + rx) / 2;
36     if (idx <= m) update(idx, new_val, 2 * v, lx, m);
37     else update(idx, new_val, 2 * v + 1, m + 1, rx);
38     tree[v] = tree[2 * v] + tree[2 * v + 1];
39 }
40
41 ll query(int l, int r, int v, int lx, int rx) {
42     if (l > rx or r < lx or l > r) return 0;
43     if (l <= lx and rx <= r) return tree[v];
44
45     int m = (lx + rx) / 2;
46     return query(l, r, 2 * v, lx, m)
47         + query(l, r, 2 * v + 1, m + 1, rx);
48 }
49
50 void update(int pos, int new_val) {
51     update(pos, new_val, 1, 0, n - 1);
52 }
53
54 ll query(int l, int r) {
55     return query(l, r, 1, 0, n - 1);
56 }
57 };

```

2.3. Fenwick tree

```

estructuras/bit.cpp
4 struct FenwickTree {
5     vector<int> bit; // binary indexed tree
6     int n;
7
8     FenwickTree(int n) {
9         this->n = n;
10        bit.assign(n, 0);
11    }
12
13    FenwickTree(vector<int> const &a) : FenwickTree(a.size()) {
14        for (size_t i = 0; i < a.size(); i++)
15            add(i, a[i]);
16    }
17
18    int sum(int r) {
19        int ret = 0;
20        for (; r >= 0; r = (r & (r + 1)) - 1)
21            ret += bit[r];
22        return ret;
23    }
24
25    int sum(int l, int r) {
26        return sum(r) - sum(l - 1);
27    }
28
29    void add(int idx, int delta) {
30        for (; idx < n; idx = idx | (idx + 1))
31            bit[idx] += delta;
32    }
33 };

```

3. Ordenamiento y Búsqueda

3.1. Two Pointers

Some uses:

Subarray sum: Given an array of n positive integers and a target sum x , and we want to find a subarray whose sum is x or report that there is no such subarray.

2SUM Problem: Given an array of n numbers and a target sum x , find two array values such that their sum is x , or report that no such values exist.

Number of Smaller: We have two arrays a and b . We want to calculate for each element b_j how many such i exist that $a_i < b_j$.

```
1 i = 0
2 for j = 0..b.size()-1:
3     while i < a.size() && a[i] < b[j]:
4         i++
5     res[j] = i
```

3.2. Búsqueda binaria en funciones monótonas

Encuentra la máxima x tal que $f(x) = \text{true}$

TIME
 $O(\log n)$

ordenamiento_busqueda/last_true.cpp ↗

```
5 int last_true(int lo, int hi, function<bool(int)> f) {
6     // if none of the values in the range work, return lo - 1
7     lo--;
8     while (lo < hi) {
9         // find the middle of the current range (rounding up)
10        int mid = lo + (hi - lo + 1) / 2;
11        if (f(mid)) {
12            // if mid works, then all numbers smaller than mid also
            // work
13            lo = mid;
14        } else {
15            // if mid does not work, greater values would not work
            // either
16            hi = mid - 1;
17        }
18    }
19    return lo;
20 }
```

Encuentra la mínima x tal que $f(x) = \text{true}$

TIME
 $O(\log n)$

ordenamiento_busqueda/first_true.cpp ↗

```
5 int first_true(int lo, int hi, function<bool(int)> f) {
6     hi++;
7     while (lo < hi) {
8         int mid = lo + (hi - lo) / 2;
9         if (f(mid)) {
10            hi = mid;
11        } else {
12            lo = mid + 1;
13        }
14    }
15    return lo;
16 }
```

4. Teoría de gráficas

4.1. Graph Traversal

4.1.1. Floodfill

Flood fill is an algorithm that identifies and labels the connected component that a particular cell belongs to in a multidimensional array.

⚠ Advertencia

Recursive implementations of flood fill sometimes lead to

- Memory limit exceeded if you run the recursive implementation on a really big grid (i.e., running the above code on a 4000×4000 grid may exceed 256 MB)
 - Non-recursive implementations generally use less memory than recursive ones.

Example (recursive implementation)

graficas/floodfill_recursive.cpp ↗

```
1 const int MAX_N = 1000;
2
3 int grid[MAX_N][MAX_N]; // the grid itself
```

```

4  int row_num;
5  int col_num;
6  bool visited[MAX_N][MAX_N]; // keeps track of which nodes have been
   visited
7  int curr_size = 0;          // reset to 0 each time we start a new
   component
8
9  void floodfill(int r, int c, int color) {
10     if ((r < 0 || r >= row_num || c < 0 || c >= col_num) // if out of
        bounds
11         || grid[r][c] != color                          // wrong
        color
12         || visited[r][c]                                // already
        visited this square
13     )
14         return;
15
16     visited[r][c] = true; // mark current square as visited
17     curr_size++;         // increment the size for each square we
        visit
18
19     // recursively call flood fill for neighboring squares
20     floodfill(r, c + 1, color);
21     floodfill(r, c - 1, color);
22     floodfill(r - 1, c, color);
23     floodfill(r + 1, c, color);
24 }
25
26 int main() {
27     /*
28     * input code and other problem-specific stuff here
29     */
30     for (int i = 0; i < row_num; i++) {
31         for (int j = 0; j < col_num; j++) {
32             if (!visited[i][j]) {
33                 curr_size = 0;
34                 /*
35                 * start a flood fill if the square hasn't already been
        visited,
36                 * and then store or otherwise use the component size
37                 * for whatever it's needed for
38                 */

```

```

39         floodfill(i, j, grid[i][j]);
40     }
41 }
42 }
43 return 0;
44 }

```

Example (iterative implementation)

graficas/floodfill_iterative.cpp 

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  const int MAX_N = 2500;
6  const int R_CHANGE[] {0, 1, 0, -1};
7  const int C_CHANGE[] {1, 0, -1, 0};
8
9  int row_num;
10 int col_num;
11 string building[MAX_N];
12 bool visited[MAX_N][MAX_N];
13
14 void floodfill(int r, int c) {
15     // Note: you can also use a queue and pop from the front for a BFS-
        based
16     // approach
17     stack<pair<int, int>> frontier;
18     frontier.push({r, c});
19     while (!frontier.empty()) {
20         r = frontier.top().first;
21         c = frontier.top().second;
22         frontier.pop();
23
24         if (r < 0 || r >= row_num || c < 0 || c >= col_num ||
            building[r][c] == '#' ||
25             visited[r][c])
26             continue;
27
28         visited[r][c] = true;
29         for (int i = 0; i < 4; i++) {

```



```

30     frontier.push({r + R_CHANGE[i], c + C_CHANGE[i]});
31 }
32 }
33 }
34
35 int main() {
36     cin >> row_num >> col_num;
37     for (int i = 0; i < row_num; i++) { cin >> building[i]; }
38
39     int room_num = 0;
40     for (int i = 0; i < row_num; i++) {
41         for (int j = 0; j < col_num; j++) {
42             if (building[i][j] == '.' && !visited[i][j]) {
43                 floodfill(i, j);
44                 room_num++;
45             }
46         }
47     }
48     cout << room_num << endl;
49 }

```

```

10
11 // Retorna el representante de la componente de "x"
12 int find(int x) {
13     return parent[x] = (parent[x] == x ? x : find(parent[x]));
14 }
15
16 // Retorna si "x" y "y" se encuentran en la misma componente
17 bool connected(int x, int y) {
18     return find(x) == find(y);
19 }
20
21 // Retorna el tamaño de la componente de "x"
22 int get_size(int x) {
23     return size[find(x)];
24 }
25
26 // Retorna la cantidad de componentes
27 int count() {
28     return components;
29 }
30
31 // Une dos componentes y retorna el nuevo representante o retorna -1
32 // si ya son parte de la misma componente
32 int merge(int x, int y) {
33     x = find(x);
34     y = find(y);
35
36     if (x == y) return -1;
37
38     components--;
39
40     if (rank[x] > rank[y]) swap(x, y);
41     parent[x] = y;
42     size[y] += size[x];
43
44     if (rank[x] == rank[y]) rank[y]++;
45     return y;
46 }
47 };

```

4.2. Disjoin Set Union

Disjoint Set Union

TIME

- **find**: $O(\log n)$
- Con path compression y union by size/rank: $O(\alpha(n))$, donde α es la inversa de la función de Ackermann (amortizada).
- Con union by size/rank, peros sin path compression: $O(\log n)$ por consulta.

graficas/dsu.cpp 

```

1 #include "setup/template.h"
2
3 struct DSU {
4     vector<int> parent, rank, size;
5     int components;
6
7     DSU(int n) : parent(n + 1), rank(n + 1, 0), size(n + 1, 1),
8                 components(n) {
9         iota(all(parent), 0);
10    }

```

4.3. Topological Sort

A topological sort of a directed acyclic graph is a linear ordering of its vertices such that for every directed edge $u \rightarrow v$ from vertex u to vertex v , u comes before v in the ordering.

DFS Version

graficas/topological_sort_dfs.cpp 

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  vector<int> top_sort;
6  vector<vector<int>> graph;
7  vector<bool> visited;
8
9  void dfs(int node) {
10     for (int next : graph[node]) {
11         if (!visited[next]) {
12             visited[next] = true;
13             dfs(next);
14         }
15     }
16     top_sort.push_back(node);
17 }
18
19 int main() {
20     int n, m; // The number of nodes and edges respectively
21     cin >> n >> m;
22
23     graph = vector<vector<int>>(n);
24     for (int i = 0; i < m; i++) {
25         int a, b;
26         cin >> a >> b;
27         graph[a - 1].push_back(b - 1);
28     }
29
30     visited = vector<bool>(n);
31     for (int i = 0; i < n; i++) {
32         if (!visited[i]) {
```

```
33         visited[i] = true;
34         dfs(i);
35     }
36 }
37 reverse(top_sort.begin(), top_sort.end());
38
39 vector<int> ind(n);
40 for (int i = 0; i < n; i++) { ind[top_sort[i]] = i; }
41
42 // Check if the topological sort is valid
43 bool valid = true;
44 for (int i = 0; i < n; i++) {
45     for (int j : graph[i]) {
46         if (ind[j] <= ind[i]) {
47             valid = false;
48             goto answer;
49         }
50     }
51 }
52 answer::;
53
54 if (valid) {
55     for (int i = 0; i < n - 1; i++) { cout << top_sort[i] + 1 << '
'; }
56     cout << top_sort.back() + 1 << endl;
57 } else {
58     cout << "IMPOSSIBLE" << endl;
59 }
60 }
```

BFS Version (Kahn's Algorithm)

graficas/topological_sort_bfs.cpp 

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6     int n, m;
7     cin >> n >> m;
```

```

8
9     vector<vector<int>> graph(n);
10    for (int i = 0; i < m; i++) {
11        int a, b;
12        cin >> a >> b;
13        graph[a - 1].push_back(b - 1);
14    }
15
16    vector<int> in_degree(n);
17    for (const vector<int> &nodes : graph) {
18        for (int node : nodes) { in_degree[node]++; }
19    }
20
21    queue<int> queue;
22    for (int i = 0; i < n; i++) {
23        if (in_degree[i] == 0) { queue.push(i); }
24    }
25    vector<int> top_sort;
26    while (!queue.empty()) {
27        int curr = queue.front();
28        queue.pop();
29        top_sort.push_back(curr);
30        for (int next : graph[curr]) {
31            if (--in_degree[next] == 0) { queue.push(next); }
32        }
33    }
34
35    if (top_sort.size() == n) {
36        for (int i = 0; i < n - 1; i++) { cout << top_sort[i] + 1 << '
'; }
37        cout << top_sort.back() + 1 << endl;
38    } else {
39        cout << "IMPOSSIBLE" << endl;
40    }
41 }

```

4.4. Detección de ciclos

Implementación para gráficas dirigidas

```

graficas/deteccion_ciclos_dirigidas.cpp
4  int n;
5  vector<vector<int>> adj;
6  vector<char> color;
7  vector<int> parent;
8  int cycle_start, cycle_end;
9
10 bool dfs(int v) {
11     color[v] = 1;
12     for (int u : adj[v]) {
13         if (color[u] == 0) {
14             parent[u] = v;
15             if (dfs(u))
16                 return true;
17         } else if (color[u] == 1) {
18             cycle_end = v;
19             cycle_start = u;
20             return true;
21         }
22     }
23     color[v] = 2;
24     return false;
25 }
26
27 void find_cycle() {
28     color.assign(n, 0);
29     parent.assign(n, -1);
30     cycle_start = -1;
31
32     for (int v = 0; v < n; v++) {
33         if (color[v] == 0 && dfs(v))
34             break;
35     }
36
37     if (cycle_start == -1) {
38         cout << "Acyclic" << endl;
39     } else {
40         vector<int> cycle;
41         cycle.push_back(cycle_start);
42         for (int v = cycle_end; v != cycle_start; v = parent[v])

```

```

43     cycle.push_back(v);
44     cycle.push_back(cycle_start);
45     reverse(cycle.begin(), cycle.end());
46
47     cout << "Cycle found: ";
48     for (int v : cycle)
49         cout << v << " ";
50     cout << endl;
51 }
52 }

```

Implementación para gráficas no dirigidas

graficas/deteccion_ciclos_no_dirigidas.cpp 

```

4  int n;
5  vector<vector<int>> adj;
6  vector<bool> visited;
7  vector<int> parent;
8  int cycle_start, cycle_end;
9
10 bool dfs(int v, int par) { // passing vertex and its parent vertex
11     visited[v] = true;
12     for (int u : adj[v]) {
13         if(u == par) continue; // skipping edge to parent vertex
14         if (visited[u]) {
15             cycle_end = v;
16             cycle_start = u;
17             return true;
18         }
19         parent[u] = v;
20         if (dfs(u, parent[u]))
21             return true;
22     }
23     return false;
24 }
25
26 void find_cycle() {
27     visited.assign(n, false);
28     parent.assign(n, -1);
29     cycle_start = -1;

```

```

30
31     for (int v = 0; v < n; v++) {
32         if (!visited[v] && dfs(v, parent[v]))
33             break;
34     }
35
36     if (cycle_start == -1) {
37         cout << "Acyclic" << endl;
38     } else {
39         vector<int> cycle;
40         cycle.push_back(cycle_start);
41         for (int v = cycle_end; v != cycle_start; v = parent[v])
42             cycle.push_back(v);
43         cycle.push_back(cycle_start);
44
45         cout << "Cycle found: ";
46         for (int v : cycle)
47             cout << v << " ";
48         cout << endl;
49     }
50 }

```

4.5. Lowest Common Ancestor (LCA)

Lowest Common Ancestor - Binary Lifting

graficas/lca.cpp 

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  // Cantidad de nodos del árbol más un margen para árboles 1-indexados
5  const int N = 3e5 + 9;
6  // Cantidad máxima de niveles ceil(log2(#nodos))
7  const int LG = 18;
8
9  // adj[u] = lista de adyacencia del nodo u
10 vector<int> adj[N];
11 // up[i][j] = 2^j-ésimo ancestro de i
12 int up[N][LG + 1];
13 // depth[i] = profundidad del nodo i

```

```

14 int depth[N];
15 // subtree_size[i] = tamaño del subárbol con raíz en i
16 int subtree_size[N];
17
18 void dfs(int u, int p = 0) {
19     up[u][0] = p;
20     depth[u] = depth[p] + 1;
21     subtree_size[u] = 1;
22
23     for (int i = 1; i <= LG; i++) {
24         up[u][i] = up[up[u][i - 1]][i - 1];
25     }
26
27     for (auto v: adj[u]) {
28         if (v != p) {
29             dfs(v, u);
30             subtree_size[u] += subtree_size[v];
31         }
32     }
33 }
34
35 // LCA de u y v
36 int lca(int u, int v) {
37     if (depth[u] < depth[v]) swap(u, v);
38
39     for (int k = LG; k >= 0; k--) {
40         if (depth[up[u][k]] >= depth[v]) u = up[u][k];
41     }
42
43     if (u == v) return u;
44
45     for (int k = LG; k >= 0; k--) {
46         if (up[u][k] != up[v][k]) u = up[u][k], v = up[v][k];
47     }
48
49     return up[u][0];
50 }
51
52 // k-ésimo ancestro de u, k >= 0
53 int kth(int u, int k) {
54     assert(k >= 0);

```

```

55     for (int i = 0; i <= LG; i++){
56         if (k & (1 << i)) u = up[u][i];
57     }
58     return u;
59 }
60
61 // Distancia entre u y v
62 int dist(int u, int v) {
63     int l = lca(u, v);
64     return depth[u] + depth[v] - (depth[l] << 1);
65 }
66
67 // k-ésimo nodo en el camino de u a v, el nodo 0 es u
68 int go(int u, int v, int k) {
69     int l = lca(u, v);
70     int d = depth[u] + depth[v] - (depth[l] << 1);
71     assert(k <= d);
72     if (depth[l] + k <= depth[u]) return kth(u, k);
73     k -= depth[u] - depth[l];
74     return kth(v, depth[v] - depth[l] - k);
75 }

```

5. Programación Dinámica

Kadane's algorithm

TIME
 $O(n)$

Given an array of integers, find the maximum sum subarray.

dp/kadane.cpp 

```

4 long long kadane(vector<long long>& arr) {
5     long long max_ending_here = arr[0];
6     long long max_so_far = arr[0];
7     for (size_t i = 1; i < arr.size(); ++i) {
8         max_ending_here = max(arr[i], max_ending_here + arr[i]);
9         max_so_far = max(max_so_far, max_ending_here);
10    }
11    return max_so_far;
12 }

```

Knapsack 0/1

TIME
 $O(nW)$

Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

dp/knapsack01.cpp 

```
4 int knapsack01(vector<int>& w, vector<int>& v, int W) {
5     int n = w.size();
6     vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
7
8
9     for (int i = 1; i <= n; ++i) {
10         for (int j = 0; j <= W; ++j) {
11             dp[i][j] = dp[i-1][j]; // no tomar el objeto i
12             if (w[i-1] <= j)
13                 dp[i][j] = max(dp[i][j], dp[i-1][j - w[i-1]] + v[i-1]);
14         }
15     }
16     return dp[n][W];
17 }
```

Coin Change

dp/coin_change.cpp 

```
4 long long coin_change(vector<int>& coins, int X) {
5     vector<long long> dp(X + 1, 0);
6     dp[0] = 1; // una forma de formar 0
7
8     for (int c : coins) {
9         for (int x = c; x <= X; ++x) {
10             dp[x] = (dp[x] + dp[x - c]) % MOD;
11         }
12     }
13     return dp[X];
14 }
```

6. Teoría de números

6.1. Cribas

Criba de primos con optimizaciones básicas

TIME
 $O(n \log \log n)$

SPACE
 $O(n)$

Find all the prime numbers in $[1, n]$. This code uses the next optimizations:

- Sieving till root
- Sieving by the odd numbers only

teoria_de_numeros/sieve_with_basic_optimizations.cpp 

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<bool> sieve(int n) {
5     vector<bool> is_prime(n + 1, true);
6     is_prime[0] = is_prime[1] = false;
7     for (int i = 3; i * i <= n; i += 2) {
8         if (is_prime[i]) {
9             for (int j = i * i; j <= n; j += 2 * i)
10                 is_prime[j] = false;
11         }
12     }
13     return is_prime;
14 }
```

Criba de divisores

Produce una criba para obtener la suma de todos los divisores de cada número en $[1, n]$ y otra para dichos divisores.

teoria_de_numeros/criba_divisores.cpp 

```
4 vector<ll> divsSum;
5 vector<vector<int>> divs;
6
7 void divisorsSieve(int n) {
8     divsSum.resize(n + 1, 0);
9     divs.resize(n + 1);
10    for (int i = 1; i <= n; ++i) {
```

```

11     for (int j = i; j <= n; j += i) {
12         divsSum[j] += i;
13         divs[j].push_back(i);
14     }
15 }
16 }

```

Criba del primo más pequeño

teoria_de_numeros/criba_primo_mas_pequeno.cpp 

```

3  vector<int> lowestPrimeSieve(int n) {
4      vector<int> lp(n + 1);
5      iota(lp.begin(), lp.end(), 0);
6      for (int i = 4; i <= n; i += 2)
7          lp[i] = 2;
8      for (int i = 3; i * i <= n; i += 2)
9          if (lp[i] == i)
10             for (int j = i * i; j <= n; j += 2 * i)
11                 lp[j] = min(lp[j], i);
12      return lp;
13 }

```

Criba del primo más grande

teoria_de_numeros/criba_primo_mas_grande.cpp 

```

3  vector<int> greatestPrimeSieve(int n)
4  {
5      vector<int> gp(n + 1);
6      iota(gp.begin(), gp.end(), 0);
7      for (int i = 2; i <= n; i++)
8          if (gp[i] == i)
9              for (int j = 2 * i; j <= n; j += i)
10                  gp[j] = i;
11      return gp;
12 }

```

6.2. Euclid's Algorithm

The algorithm is based on the formula

$$\gcd(a, b) = \begin{cases} a & b = 0 \\ \gcd(b, a \bmod b) & b \neq 0 \end{cases}$$

6.2.1. Extended Euclid's Algorithm:

Euclid's algorithm can also be extended so that it gives integers x and y for which

$$ax + by = \gcd(a, b)$$

Iterative version

TIME
 $O(\log \min(a, b))$

teoria_de_numeros/extended_euclid_algorithm.cpp 

```

1  int gcd(int a, int b, int& x, int& y) {
2      x = 1, y = 0;
3      int x1 = 0, y1 = 1, a1 = a, b1 = b;
4      while (b1) {
5          int q = a1 / b1;
6          tie(x, x1) = make_tuple(x1, x - q * x1);
7          tie(y, y1) = make_tuple(y1, y - q * y1);
8          tie(a1, b1) = make_tuple(b1, a1 - q * b1);
9      }
10     return a1;
11 }

```

6.3. Euler's Theorem

Euler's totient function $\varphi(n)$ gives the number of integers between $1, \dots, n$ that are coprime to n .

Any value of $\varphi(n)$ can be calculated from the prime factorization of n using the formula

$$\varphi(n) = \prod_{i=1}^k p_i^{\alpha_i - 1} (p_i - 1)$$

Teorema 6.3.1 (Euler's theorem)

For all positive coprime integers x and m

$$x^{\varphi(m)} \bmod m = 1$$

6.4. Solving Equations

We can efficiently solve a Diophantine equation by using the extended Euclid's algorithm which gives integers x and y that satisfy the equation

$$ax + by = \gcd(a, b)$$

A Diophantine equation can be solved exactly when c is divisible by $\gcd(a, b)$.

If a pair (x, y) is a solution, then also all pairs

$$\left(x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)}\right)$$

are solutions, where k is any integer.

7. Combinatoria

7.1. Binomial Coefficients

Nota

For $n < k$ the value of $\binom{n}{k}$ is assumed to be zero.

Basic Improved Implementation

```
1 int C(int n, int k) {
2     double res = 1;
3     for (int i = 1; i <= k; ++i)
4         res = res * (n - k + i) / i;
5     return (int)(res + 0.01);
6 }
```

Table of binomial coefficients using Pascal's Triangle identity

```
1 const int maxn = ...;
2 int C[maxn + 1][maxn + 1];
3 C[0][0] = 1;
4 for (int n = 1; n <= maxn; ++n) {
5     C[n][0] = C[n][n] = 1;
6     for (int k = 1; k < n; ++k)
7         C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
8 }
```

8. Strings

8.1. Trie

Estructura

strings/trie.cpp

```
4  const int K = 26;
5
6  struct Vertex {
7      int next[K];
8      bool output = false;
9
10     Vertex() {
11         fill(begin(next), end(next), -1);
12     }
13 };
14
15 vector<Vertex> trie(1);
```

Agregar una cadena

strings/trie.cpp

```
19 void add_string(string const& s) {
20     int v = 0;
21     for (char ch : s) {
22         int c = ch - 'a';
23         if (trie[v].next[c] == -1) {
24             trie[v].next[c] = trie.size();
25             trie.emplace_back();
26         }
27         v = trie[v].next[c];
28     }
29     trie[v].output = true;
30 }
```

8.2. String hashing

hash-doble


```

3  struct Hash { //doble hashing
4      vector<ll> h1, h2;
5      vector<ll> p1, p2;
6      ll p=31; //tomamos esta p para evitar colisiones
7      Hash(string &s) {
8          int n = s.size();
9          h1.resize(n+1, 0);
10         h2.resize(n+1, 0);
11         p1.resize(n+1, 1);
12         p2.resize(n+1, 1);
13         //hacemos doble hasheo para evitar colisiones
14         for (int i = 0; i < n; i++) {
15             p1[i+1] = (p1[i] * p) % MOD; //a uno le aplicamos modulo
16             p2[i+1] = (p2[i] * p); //el otro ocupamos el modulo
17             //implicito de ll
18             //uno con mod 10e9+7 y otro con 2e64 que es LLONG_MAX
19             int val = s[i] - 'a' + 1; //hasheamos con el ascii
20             h1[i+1] = (h1[i] * p + val) % MOD;
21             h2[i+1] = (h2[i] * p + val);
22         }
23     }
24
25     pair<ll,ll> get(ll l, ll r) { //comparamos los 2 hashing
26         ll x1 = (h1[r+1] - h1[l] * p1[r-l+1] % MOD + MOD) % MOD;
27         ll x2 = (h2[r+1] - h2[l] * p2[r-l+1]);
28         return {x1, x2};
29     }
30 };
31 Hash hs(s); //inicializar
32 hs.get(i,j); //lugar de sonde se compara
33 if(hs.get(l,m)==hs.get(i,j)) //si 2 cadenas son iguales

```

8.3. KMP – Knuth-Morris-Pratt

KMP – Knuth-Morris-Pratt

TIME
 $O(n + m)$

SPACE
 $O(m)$

Cuenta ocurrencias de pat en txt. buildPhi construye el arreglo de fallos: phi[j] = prefijo propio más largo de pat[0..j] que es sufijo. Al fallar retrocede a phi[i-1] en lugar de reiniciar.

```

5  vll buildPhi(const vll& pat) {
6      ll m = pat.size();
7      vll phi(m, 0);
8      for (ll j = 1, i = 0; j < m; j++) {
9          while (i > 0 && pat[i] != pat[j]) i = phi[i - 1];
10         if (pat[i] == pat[j]) i++;
11         phi[j] = i;
12     }
13     return phi;
14 }
15
16 ll KMP(const vll& pat, const vll& txt) {
17     ll m = pat.size(), n = txt.size();
18     if (m == 0) return 0;
19     vll phi = buildPhi(pat);
20     ll matches = 0;
21     for (ll i = 0, j = 0; j < n; j++) {
22         while (i > 0 && pat[i] != txt[j]) i = phi[i - 1];
23         if (pat[i] == txt[j]) i++;
24         if (i == m) { matches++; i = phi[i - 1]; }
25     }
26     return matches;
27 }

```

Bordes de un string

TIME
 $O(n)$

SPACE
 $O(n)$

Dado phi de un string, devuelve todas las longitudes de bordes no triviales (prefijo = sufijo, distinto del string completo), de mayor a menor. Se obtiene siguiendo la cadena $\text{phi}[n-1]$, $\text{phi}[\text{phi}[n-1]-1]$, ... hasta llegar a 0.

strings/Borders_KMP.cpp

```
6  vll getBorders(const vll& phi) {
7      ll n = phi.size();
8      vll borders;
9      for (ll i = phi[n - 1]; i; i = phi[i - 1])
10         borders.push_back(i);
11     return borders;
12 }
```

Conteo de ocurrencias de cada prefijo

TIME
 $O(n)$

SPACE
 $O(n)$

Dado phi de un string, $\text{cnt}[L]$ = cuántas veces aparece el prefijo de longitud L como substring dentro de todo el string (incluyendo su propia ocurrencia como prefijo). Se calcula acumulando conteos por valor de phi y propagándolos hacia abajo en la cadena de bordes.

strings/Occurrences_KMP.cpp

```
6  vll countPrefixOccurrences(const vll& phi) {
7      ll n = phi.size();
8      vll cnt(n + 1, 0);
9      for (ll i = 0; i < n; i++)
10         cnt[phi[i]]++;
11     for (ll i = n - 1; i > 0; i--)
12         cnt[phi[i - 1]] += cnt[i];
13     for (ll i = 0; i <= n; i++)
14         cnt[i]++;
15     return cnt;
16 }
```

9. Manipulación de bits

9.1. Máscara de bits

9.1.1. Manipulación individual de bits

Establece en 1 el k -ésimo bit de x :

```
1 #define ON(x, k) ((x) | (1LL << (k)))
```

Establece en 0 el k -ésimo bit de x :

```
1 #define OFF(x, k) ((x) & ~(1LL << (k)))
```

Invierte el k -ésimo bit de x :

```
1 #define TOGGLE(x, k) ((x) ^ (1LL << (k)))
```

Comprueba si el k -ésimo bit de x es 1:

```
1 #define CHECK(x, k) ((x) & (1LL << (k)))
```

9.1.2. Working with the rightmost 1-bit

- $x \& -x$: Isolates the rightmost set bit (keeps only the least significant bit that is 1, sets all others to 0).
- $x \& (x - 1)$: Removes the rightmost set bit from x (turns the rightmost 1 to 0).
- $x | (x - 1)$: Sets all trailing zeros of x to 1.

9.1.3. Creating useful bit masks

$(1 \ll n) - 1$: Creates a bit mask with the first n bits set to 1.

9.1.4. Common bit manipulation tricks

- A positive number x is a power of two exactly when $x \& (x - 1) = 0$.
- $x \wedge A \wedge B$: Determines which is not the current value of x between A and B .

9.1.5. Bit operations as math alternatives

- $x \ll k$: equivalent to $x * \text{pow}(2, k)$
- $x \gg k$: equivalent to $x / \text{pow}(2, k)$ (integer division)
- $x \& ((1 \ll k) - 1)$: equivalent to $x \% \text{pow}(2, k)$
- $A + B$: equivalent to $(A \wedge B) + 2 * (A \& B)$ or $(A | B) + (A \& B)$

9.1.6. GNU C++ compiler built-in functions

__builtin_clz(x): the number of leading zeros (zeros on the left side of the bit representation).

__builtin_ctz(x): the number of trailing zeros (zeros on the right side of the bit representation).

__builtin_popcount(x): the number of ones in the bit representation.

__builtin_parity(x): the parity (even or odd) of the number of ones in the bit representation.

Example of use

```
1 int x = 5328; // 000000000000000001010011010000
2 cout << __builtin_clz(x) << "\n"; // 19
3 cout << __builtin_ctz(x) << "\n"; // 4
4 cout << __builtin_popcount(x) << "\n"; // 5
5 cout << __builtin_parity(x) << "\n"; // 1
```

Nota

The above functions only support **int** numbers, but there are also **long long** versions of the functions available with the suffix **ll** (e.g. **__builtin_popcountll(x)**).

9.2. Representing Sets

Every subset of a set $\{0, 1, 2, \dots, n-1\}$ can be represented as an n bit integer whose one bits indicate which elements belong to the subset.

Goes through the subsets with exactly k elements

```
1 for (int b = 0; b < (1<<n); b++) {
2     if (__builtin_popcount(b) == k) {
3         // process subset b
4     }
5 }
```

Goes through the subsets of a set x

```
1 int b = 0;
2 do {
3     // process subset b
4 } while (b=(b-x)&x);
```

The idea is that the formula $b - x$ detects the rightmost one bit in x that is zero in b . This bit becomes one and all bits after it become zero. Then the and operation ensures

that the resulting value is a subset of x . Note that $b - x$ equals $-(x - b)$ so we can think that we first remove all one bits that appear in b and then invert the value and add one.

10. Matemáticas

10.1. Operaciones básicas con matrices

Operaciones básicas con matrices

matematicas/matrix_exponentiation.cpp

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 const int mod = 998244353;
5
6 struct Mat {
7     int n, m;
8     vector<vector<int>> a;
9     Mat() { }
10    Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
11    Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0; a = v; }
12    inline void make_unit() {
13        assert(n == m);
14        for (int i = 0; i < n; i++) {
15            for (int j = 0; j < n; j++) a[i][j] = i == j;
16        }
17    }
18    inline Mat operator + (const Mat &b) {
19        assert(n == b.n && m == b.m);
20        Mat ans = Mat(n, m);
21        for(int i = 0; i < n; i++) {
22            for(int j = 0; j < m; j++) {
23                ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
24            }
25        }
26        return ans;
27    }
28    inline Mat operator - (const Mat &b) {
```

```

29  assert(n == b.n && m == b.m);
30  Mat ans = Mat(n, m);
31  for(int i = 0; i < n; i++) {
32      for(int j = 0; j < m; j++) {
33          ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
34      }
35  }
36  return ans;
37 }
38 inline Mat operator * (const Mat &b) {
39     assert(m == b.n);
40     Mat ans = Mat(n, b.m);
41     for(int i = 0; i < n; i++) {
42         for(int j = 0; j < b.m; j++) {
43             for(int k = 0; k < m; k++) {
44                 ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] %
45                     mod) % mod;
46             }
47         }
48     }
49     return ans;
50 }
51 inline Mat pow(long long k) {
52     assert(n == m);
53     Mat ans(n, n), t = a; ans.make_unit();
54     while (k) {
55         if (k & 1) ans = ans * t;
56         t = t * t;
57         k >>= 1;
58     }
59     return ans;
60 }
61 inline Mat& operator += (const Mat& b) { return *this = (*this) +
62     b; }
63 inline Mat& operator -= (const Mat& b) { return *this = (*this) -
64     b; }
65 inline Mat& operator *= (const Mat& b) { return *this = (*this) *
66     b; }
67 inline bool operator == (const Mat& b) { return a == b.a; }
68 inline bool operator != (const Mat& b) { return a != b.a; }
69 };

```

```

66
67 int32_t main() {
68     ios_base::sync_with_stdio(0);
69     cin.tie(0);
70     int n; long long k; cin >> n >> k;
71     Mat a(n, n);
72     for (int i = 0; i < n; i++) {
73         for (int j = 0; j < n; j++) {
74             cin >> a.a[i][j];
75         }
76     }
77     Mat ans = a.pow(k);
78     for (int i = 0; i < n; i++) {
79         for (int j = 0; j < n; j++) {
80             cout << ans.a[i][j] << ' ';
81         }
82         cout << '\n';
83     }
84     return 0;
85 }
86 // https://judge.yosupo.jp/problem/pow_of_matrix

```

11. Teoría

11.1. Teoría de números

11.1.1. Algunas funciones relevantes

- La función contadora de primos $\pi(n)$ da el *número de primos hasta n* . Esta función es asintótica a $\frac{n}{\ln(n)}$

x	10	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9
$\pi(x)$	4	25	168	1,229	9,592	78,498	664,579	5,761,455	50,847,534

- Número de divisores de un entero n :

$$\tau(n) = \prod_{i=1}^k (\alpha_i + 1)$$

- Un **número altamente compuesto** es un entero positivo que tiene más divisores que cualquier entero positivo menor que él.

- Un **número altamente compuesto superior** es un número natural que, en un sentido riguroso y específico, posee una gran cantidad de divisores.

Algunos números altamente compuestos superiores son:

n	6	60	360	5040	55440	720720	4324320	21621600
$\tau(n)$	4	12	24	60	120	240	384	576

n	367567200	6983776800	13967553600	321253732800
$\tau(n)$	1152	2304	2688	5376

El número $18401055938125660800 \approx 2 \times 10^{18}$ es altamente compuesto y posee 184320 divisores.

Para números hasta 10^{88} se cumple que $\tau(n) < 3.6\sqrt[3]{n}$.

- Suma de divisores de un entero n :

$$\sigma(n) = \prod_{i=1}^k (1 + p_i + \dots + p_i^{\alpha_i}) = \prod_{i=1}^k \frac{p_i^{\alpha_i+1} - 1}{p_i - 1}$$

- Producto de divisores de un entero n :

$$n^{\frac{\tau(n)}{2}}$$

11.1.2. Aritmética modular

Definición 11.1.1 (Modular multiplicative inverse)

The *modular multiplicative inverse* of x with respect to m is a value $\text{inv}_m(x)$ such that

$$x \cdot \text{inv}_m(x) \bmod m = 1$$

If m is prime

$$\text{inv}_m(x) = x^{m-2}$$

- Factorial inverso modular: $\text{inv}_p((x-1)!) \equiv (x \cdot \text{inv}_p(x!)) \bmod p$

11.2. Combinatoria

11.2.1. Números de Fibonacci

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

11.2.2. Números de Catalán

Fórmula recursiva:

$$C_0 = C_1 = 1$$

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}, \quad n \geq 2$$

Fórmulas analíticas:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!} = \prod_{k=2}^n \frac{n+k}{k}, \quad n \geq 0$$

11.2.2.1. Aplicaciones

El número de Catalán C_n es la solución para:

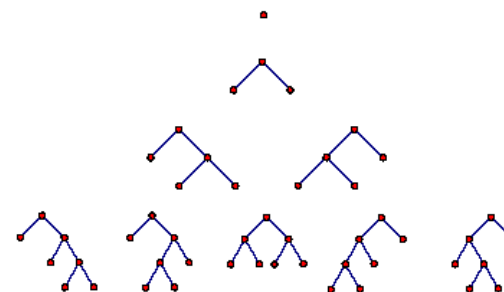
- Número de secuencias de paréntesis correctas formadas por n paréntesis de apertura y n paréntesis de cierre.

((())) ()(()) ()()() (()()) ...

- Las aplicaciones sucesivas de un operador binario pueden representarse mediante un árbol binario completo. Un árbol binario enraizado es completo si cada vértice tiene exactamente dos hijos o ninguno. De esto se sigue que C_n es el número de árboles binarios completos enraizados con $n+1$ hojas (los vértices no están numerados).

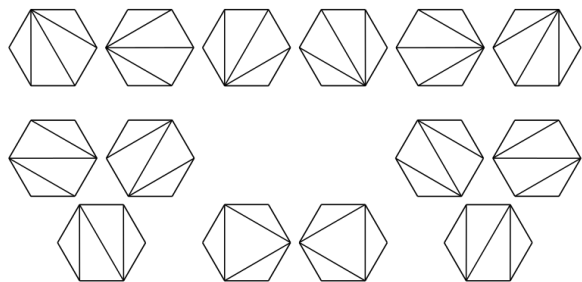
Ejemplo. Para $n+1 = 4$ factores a, b, c, d :

$$a(b(cd)) \quad a((bc)d) \quad (ab)(cd) \quad ((ab)c)d \quad (a(bc))d$$



- El número de triangulaciones de un polígono convexo con $n+2$ lados (es decir, el número de particiones del polígono en triángulos disjuntos usando diagonales).

Ejemplo. Caso para $n = 4$



- El número de formas de conectar los $2n$ puntos de una circunferencia para formar n cuerdas disjuntas.
- El número de árboles binarios completos no isomorfos con n nodos internos (es decir, nodos que tienen al menos un hijo).
- El número de caminos monótonos en una retícula desde el punto $(0, 0)$ hasta el punto (n, n) en una cuadrícula de tamaño $n \times n$, que no pasan por encima de la diagonal principal (i.e. conectando $(0, 0)$ a (n, n)).
- El número de permutaciones de longitud n que pueden ordenarse con una pila (stack-sortable permutation). Se puede demostrar que la reordenación es ordenable con una pila si y solo si no existe un índice $i < j < k$ tal que $a_k < a_i < a_j$.
- El número de particiones no cruzadas de un conjunto de n elementos.
- El número de formas de cubrir la escalera $1, \dots, n$ usando n rectángulos. La escalera consta de n columnas, donde la i -ésima columna tiene altura i .

11.2.3. Identidades binomiales

- Pascal's Triangle:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

- Symmetry rule:

$$\binom{n}{k} = \binom{n}{n-k}$$

- Factoring in:

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$$

- Sum over k :

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

- Sum over n :

$$\sum_{m=0}^n \binom{m}{k} = \binom{n+1}{k+1}$$

- Sum over n and k :

$$\sum_{k=0}^m \binom{n+k}{k} = \binom{n+m+1}{m}$$

- Sum of the squares:

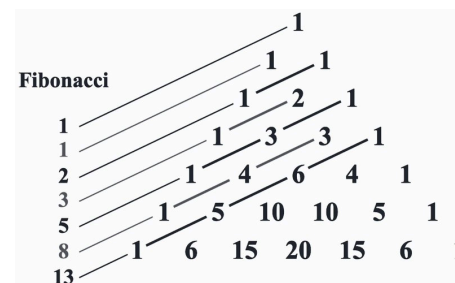
$$\binom{n}{0}^2 + \binom{n}{1}^2 + \dots + \binom{n}{n}^2 = \binom{2n}{n}$$

- Weighted sum:

$$1 \binom{n}{1} + 2 \binom{n}{2} + \dots + n \binom{n}{n} = n 2^{n-1}$$

- Connection with the Fibonacci numbers:

$$\binom{n}{0} + \binom{n-1}{1} + \dots + \binom{n-k}{k} + \dots + \binom{0}{n} = F_{n+1}$$



11.2.4. Estrellas y barras

Teorema 11.2.1

El número de formas de colocar n objetos idénticos en k cajas etiquetadas es

$$\binom{n+k-1}{n}$$

Utilizando este resultado podemos contar el número de sumas de k enteros acotados inferiormente, i.e, el número de soluciones para la ecuación

$$x_1 + x_2 + \dots + x_k = n$$

con $x_i \geq a_i$.

La solución es:

$$\binom{n - (a_1 + a_2 + \dots + a_k) + k - 1}{n}$$

Cuando los x_i están acotados superiormente, podemos usar el principio de inclusión-exclusión para contar el número de soluciones.

11.2.5. Principio de inclusión-exclusión

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{i=1}^n |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| + \dots + (-1)^{n-1} |A_1 \cap A_2 \cap \dots \cap A_n|$$

Forma compacta:

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{\emptyset \neq J \subseteq \{1, 2, \dots, n\}} (-1)^{|J|-1} \left| \bigcap_{j \in J} A_j \right|$$

11.2.6. Derangement (subfactorial)

Un desarreglo (derangement) es una permutación que no tiene puntos fijos. Sea d_n el número de desarreglos de una secuencia $1 \dots n$. Se tiene la recurrencia $d_n = (n - 1)(d_{n-1} + d_{n-2})$. Además, d_n es el entero más cercano a $\frac{n!}{e}$.

$$d_n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$$

$$D_n = \sum_{i=0}^n (-1)^i \binom{n}{i} (n-i)! \approx \frac{n!}{e}$$

$$D_n = \begin{cases} 0 & \text{si } n = 1 \\ 1 & \text{si } n = 2 \\ (n-1)(D_{n-2} + D_{n-1}) & \text{si } n \geq 3 \end{cases}$$

$$D_n = nD_{n-1} + (-1)^n \text{ para } n \geq 1$$

11.3. Primeros términos de algunas sucesiones básicas

n	p_n	F_n	C_n	2^n	$n!$	$!n$
0	—	0	1	1	1	1
1	2	1	1	2	1	0
2	3	1	2	4	2	1
3	5	2	5	8	6	2
4	7	3	14	16	24	9
5	11	5	42	32	120	44
6	13	8	132	64	720	265
7	17	13	429	128	5040	1854
8	19	21	1430	256	40320	14833
9	23	34	4862	512	362880	133496
10	29	55	16796	1024	3628800	1334961
11	31	89	58786	2048	39916800	14684570
12	37	144	208012	4096	479001600	176432560
13	41	233	742900	8192	6227020800	2290792932
14	43	377	2674440	16384	87178291200	32071101049
15	47	610	9694845	32768	1307674368000	481066515734

11.4. Geometría

11.4.1. Ternas pitagóricas

$$a^2 + b^2 = c^2$$

Every Pythagorean triple (a, b, c) is generated uniquely by

$$a = k(m^2 - n^2), b = k(2mn), c = k(m^2 + n^2)$$

where m, n , and k are positive integers with $m > n$, and with m and n coprime and not both odd.

11.4.2. Trigonometría

Sean a, b, c las longitudes de los lados y α, β, γ sus ángulos opuestos, entonces:

Semiperímetro:

$$s = \frac{a + b + c}{2}$$

Área:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

Circunradio:

$$R = \frac{abc}{4A}$$

Inradio:

$$r = \frac{A}{s}$$

Longitud de la mediana (divide el triángulo en dos triángulos de igual área):

$$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$$

Longitud de la bisectriz (divide los ángulos en dos):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b+c} \right)^2 \right]}$$

Ley de senos:

$$\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$$

Ley de cosenos:

$$a^2 = b^2 + c^2 - 2bc \cos \alpha$$

Ley de tangentes:

$$\frac{a+b}{a-b} = \frac{\tan\left(\frac{\alpha+\beta}{2}\right)}{\tan\left(\frac{\alpha-\beta}{2}\right)}$$

11.4.2.1. Suma de ángulos

$$\sin(\alpha \pm \beta) = \sin \alpha \cos \beta \pm \cos \alpha \sin \beta$$

$$\cos(\alpha \pm \beta) = \cos \alpha \cos \beta \mp \sin \alpha \sin \beta$$

$$\tan(\alpha \pm \beta) = \frac{\tan \alpha \pm \tan \beta}{1 \mp \tan \alpha \tan \beta}$$

11.4.2.2. Transformación de suma a producto

$$\sin \alpha \pm \sin \beta = 2 \sin \frac{\alpha \pm \beta}{2} \cos \frac{\alpha \mp \beta}{2}$$

$$\cos \alpha + \cos \beta = 2 \cos \frac{\alpha + \beta}{2} \cos \frac{\alpha - \beta}{2}$$

$$\cos \alpha - \cos \beta = -2 \sin \frac{\alpha + \beta}{2} \sin \frac{\alpha - \beta}{2}$$

$$\tan \alpha \pm \tan \beta = \frac{\sin(\alpha \pm \beta)}{\cos \alpha \cos \beta}$$

11.4.3. Producto punto

$$\vec{a} \cdot \vec{b} = ab \cos(\theta)$$

- El producto punto es positivo si el ángulo entre ellos es agudo.
- Es negativo si el ángulo es obtuso.
- Es cero si son ortogonales (forman un ángulo recto).

11.4.3.1. Propiedades

1. Norma al cuadrado de $\vec{a}$:

$$|\vec{a}|^2 = \vec{a} \cdot \vec{a}$$

2. Longitud de $\vec{a}$:

$$|\vec{a}| = \sqrt{\vec{a} \cdot \vec{a}}$$

3. Proyección de $\vec{b}$ sobre $\vec{a}$:

$$\text{proj}_{\vec{a}} \vec{b} = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}|^2} \vec{a}$$

4. Ángulo entre vectores:

$$\arccos\left(\frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|}\right)$$

11.5. Sumas y series

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4} = \left(\sum_{i=1}^n i\right)^2 \quad \sum_{i=1}^n i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$\sum_{k=a}^b c^k = \frac{c^{b+1} - c^a}{c - 1}, \quad c \neq 1$$

$$g_k(n) = \sum_{i=1}^n i^k = \frac{1}{k+1} \left(n^{k+1} + \sum_{j=1}^k \binom{k+1}{j+1} (-1)^{j+1} g_{k-j}(n) \right)$$

$$\sum_{i=0}^n ic^i = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}, \quad c \neq 1$$

$$\sum_{i=0}^{\infty} ic^i = \frac{c}{(1-c)^2}, \quad |c| < 1$$

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, -\infty < x < \infty$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, -1 < x \leq 1$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, -1 \leq x \leq 1$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, -\infty < x < \infty$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, -\infty < x < \infty$$

12. Otros

12.1. Estructura para fracciones

Estructura para representar fracciones con operaciones básicas.

otros/fraccion.cpp 

```
1  #include "../setup/template.h"
2
3  struct fraccion{
4      ll num, den;
5      fraccion(){
6          num = 0, den = 1;
7      }
8      fraccion(ll x, ll y){
9          if(y < 0)
```

```
10         x *= -1, y *= -1;
11         ll d = __gcd(abs(x), abs(y));
12         num = x/d, den = y/d;
13     }
14     fraccion(ll v){
15         num = v;
16         den = 1;
17     }
18     fraccion operator+(const fraccion& f) const{
19         ll d = __gcd(den, f.den);
20         return fraccion(num*(f.den/d) + f.num*(den/d), den*(f.den/
21         d));
22     }
23     fraccion operator-() const{
24         return fraccion(-num, den);
25     }
26     fraccion operator-(const fraccion& f) const{
27         return *this + (-f);
28     }
29     fraccion operator*(const fraccion& f) const{
30         return fraccion(num*f.num, den*f.den);
31     }
32     fraccion operator/(const fraccion& f) const{
33         return fraccion(num*f.den, den*f.num);
34     }
35     fraccion operator+=(const fraccion& f){
36         *this = *this + f;
37         return *this;
38     }
39     fraccion operator-=(const fraccion& f){
40         *this = *this - f;
41         return *this;
42     }
43     fraccion operator++(int xd){
44         *this = *this + 1;
45         return *this;
46     }
47     fraccion operator--(int xd){
48         *this = *this - 1;
49         return *this;
50     }
```

```

50     fraccion operator*=(const fraccion& f){
51         *this = *this * f;
52         return *this;
53     }
54     fraccion operator/=(const fraccion& f){
55         *this = *this / f;
56         return *this;
57     }
58     bool operator==(const fraccion& f) const{
59         ll d = __gcd(den, f.den);
60         return (num*(f.den/d) == (den/d)*f.num);
61     }
62     bool operator!=(const fraccion& f) const{
63         ll d = __gcd(den, f.den);
64         return (num*(f.den/d) != (den/d)*f.num);
65     }
66     bool operator >(const fraccion& f) const{
67         ll d = __gcd(den, f.den);
68         return (num*(f.den/d) > (den/d)*f.num);
69     }
70     bool operator <(const fraccion& f) const{
71         ll d = __gcd(den, f.den);
72         return (num*(f.den/d) < (den/d)*f.num);
73     }
74     bool operator >=(const fraccion& f) const{
75         ll d = __gcd(den, f.den);
76         return (num*(f.den/d) >= (den/d)*f.num);
77     }
78     bool operator <=(const fraccion& f) const{
79         ll d = __gcd(den, f.den);
80         return (num*(f.den/d) <= (den/d)*f.num);
81     }
82     fraccion inverso() const{
83         return fraccion(den, num);
84     }
85     fraccion fabs() const{
86         fraccion nueva;
87         nueva.num = abs(num);
88         nueva.den = den;
89         return nueva;
90     }

```

```

91     double value() const{
92         return (double)num / (double)den;
93     }
94     string str() const{
95         stringstream ss;
96         ss << num;
97         if(den != 1) ss << "/" << den;
98         return ss.str();
99     }
100 };
101
102 ostream &operator<<(ostream &os, const fraccion & f) {
103     return os << f.str();
104 }
105
106 istream &operator>>(istream &is, fraccion & f){
107     ll num = 0, den = 1;
108     string str;
109     is >> str;
110     size_t pos = str.find("/");
111     if(pos == string::npos){
112         istringstream(str) >> num;
113     }else{
114         istringstream(str.substr(0, pos)) >> num;
115         istringstream(str.substr(pos + 1)) >> den;
116     }
117     f = fraccion(num, den);
118     return is;
119 }

```